LABORATORIO DI ELETTRONICA

IMPARA E SPERIMENTA

VOLUME 3



ELAB

Indice

Capitolo 1- Un viaggio nella storia della programmazione 05

Capitolo 2 - Cos'è la Programmazione?.....09

Capitolo 3 - Ardware e Software: cosa cambia? 13

Capitolo 4 - Il collegamento tra te e Arduino (IDE)..... 37

Capitolo 5 - Come è fatto un programma Arduino?..... 47

Capitolo 6 - I pin digitali: le dita di Arduino..... 53

Capitolo 7 - I pin imput e output..... 63

Capitolo 8 - I PIN ANALOGICI: ARDUINO IMPARA A SENTIRE LE QUANTITÀ..... 75

Capitolo 9 - 85

Capitolo 10 - 93

Capitolo 11 - 103

Capitolo 12 - 109

Non è finita qui! 114

COMPONENTI DEL KIT

1x Breadboard 830 punti
1x Multimetro + puntalini a coccodrillo
Cavi di differenti lunghezze
1x Clip Batteria intestata con header
Resistori 4 Bande:
- 10x 100 Ohm
- 10x 220 Ohm
- 10x 330 Ohm
- 10x 470 Ohm
- 10x 1 kOhm

LEDs:
- 5x Blu
- 5x Rosso
- 5x Verde
- 5x Giallo
- 5x Bianco
- 5x RGB catodo comune

5x Pulsanti
4x Transistor
4x Diodi
4x Fotosensori
4x Condensatori da 1000uF
2x Motori a corrente continua
2x Ruote

CAPITOLO 1

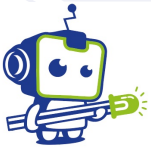
Un viaggio nella storia della programmazione

Bentornati, giovani esploratori! Oggi vi porterò a scoprire uno dei poteri più incredibili dell'essere umano: il linguaggio con cui dà ordini alle macchine.» Programmare significa spiegare a una macchina cosa deve fare, passo dopo passo. Ma prima di iniziare a scrivere codice, facciamo un salto indietro nel tempo per capire come tutto è cominciato.

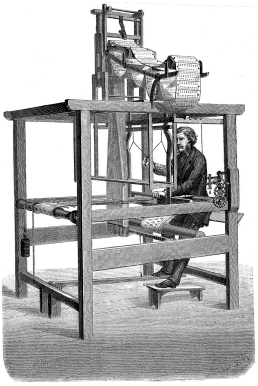
Prima dei computer: le idee geniali.



Siamo a metà dell'Ottocento. Non esistono computer, non esiste l'elettricità domestica e molte invenzioni moderne devono ancora nascere. Eppure una giovane matematica inglese, Ada Lovelace, immagina che una macchina possa seguire istruzioni per eseguire calcoli complessi. È la prima persona a descrivere quello che oggi chiamiamo algoritmo: una sequenza di passi chiara e ordinata che permette a una macchina di fare qualcosa.



È come se avesse visto il futuro un secolo prima!



Qualche anno prima, nel 1804, Joseph Jacquard aveva già costruito un telaio che funzionava tramite schede forate. Ogni scheda diceva alla macchina come muovere i fili per creare un certo disegno. Anche se era solo un meccanismo, possiamo considerarlo uno dei primi esempi di "programma".

I primi computer: giganti lenti e rumorosi

Molto tempo dopo, durante la seconda guerra mondiale, nascono i primi computer elettronici.

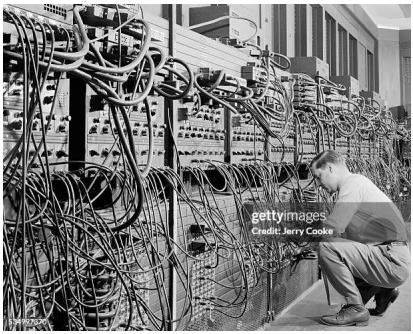
Erano enormi, lenti, rumorosi e pieni di valvole che si scaldavano subito. Uno dei più famosi è ENIAC, costruito nel 1946: era grande come un'aula scolastica. Per programmarlo non si usava una tastiera, ma bisognava collegare cavi, spostare interruttori e modificare interi pannelli.

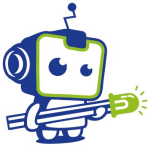
Il linguaggio macchina

Con l'arrivo dei primi sistemi più avanzati, i programmatori iniziano a scrivere sequenze di numeri (0 e 1) che rappresentano istruzioni per il computer.
È il livello più "puro" e difficile della programmazione.

Da 0 e 1 alle parole vere

Negli anni '50 e '60 succede qualcosa di rivoluzionario: nascono i primi linguaggi di programmazione leggibili dalle persone.





Altro che tastiera... programmare allora voleva dire diventare elettricisti!

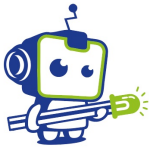
FORTRAN viene usato per la matematica, COBOL per il mondo delle aziende e BASIC per chi vuole imparare a programmare. Per la prima volta è possibile scrivere comandi simili all'inglese, molto più facili da ricordare e da leggere.

Il boom del personal computer

Ogni scoperta ha portato a nuove invenzioni, che hanno cambiato il mondo e il nostro modo di vivere. E chissà quali altre incredibili scoperte ci riserverà il futuro!

Programmare oggi: da Arduino all'intelligenza artificiale

Oggi programmare è alla portata di tutti. Anche una piccola scheda come Arduino permette di creare luci intelligenti, piccoli robot, sistemi di allarme e giochi interattivi. Nei prossimi capitoli imparerai a usarla passo dopo passo.



Programmare non significa solo parlare a un computer: significa creare qualcosa che prima non esisteva.

NOTE

- Una istruzione è un comando dato a una macchina.
- Un programma è un insieme ordinato di istruzioni.
- I linguaggi di programmazione servono per facilitare la comunicazione tra noi e il computer, senza usare 0 e 1.

NOTE

Cos'è la programmazione?

La programmazione è il modo in cui comunichiamo con i dispositivi elettronici programmabili per dir loro cosa devono fare. È un po' come scrivere una ricetta che deve essere seguita passo dopo passo. Immagina di voler preparare una torta: hai una lista di ingredienti e una serie di passaggi da seguire nell'ordine corretto.

Se rispetti ogni passaggio, la torta verrà fuori!

La programmazione funziona esattamente allo stesso modo: se riusciamo a dire, in modo chiaro e preciso, cosa deve fare l'oggetto che vogliamo programmare, possiamo creare cose meravigliose. Anche i videogiochi, le app sullo smartphone e i software che usi ogni giorno sono programmati!

I linguaggi di programmazione

I dispositivi elettronici programmabili, proprio come gli esseri umani, comunicano attraverso linguaggi diversi. Nel loro caso si chiamano linguaggi di programmazione.

E come qualunque lingua, hanno delle regole grammaticali ben precise: se non le rispettiamo, la macchina non capisce e il programma non funziona.

Prova a leggere questa frase:

Io può volere andare ma se io volere andare come fare per volere andare ad andare?

Confusa, vero?

Lo stesso succede quando si scrive male un programma.

Nel mondo della programmazione, ad "esaminare" se ciò che abbiamo scritto è corretto, c'è una sorta di professore molto severo:

il compilatore.

Il suo compito è controllare se il codice rispetta le regole del linguaggio. Se è corretto "ti dà la sufficienza" e fa partire il programma; se invece ci sono errori, non ti lascia proseguire e non compila.

Nei linguaggi di programmazione non usiamo frasi, ma istruzioni, chiamate anche linee di codice.

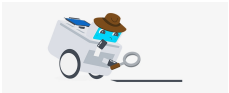
Ogni linea di codice corrisponde a un'azione.

Per esempio, se dovessimo programmare un robot per farlo muovere, potremmo scrivere:



Per esempio, se dovessimo programmare un robot per farlo muovere, potremmo scrivere:

- Vai avanti di 3 passi
- Girati a destra
- Saluta



E il robot eseguirà queste azioni esattamente nel loro ordine.

Un esempio pratico

Prova a fare un piccolo esperimento con un tuo amico. Immagina di doverlo guidare all'uscita della scuola. Scrivi tutte le istruzioni, una per una:

1. Alzati dalla sedia
2. Fai un passo in avanti.
3. Girati di 90 gradi a destra.

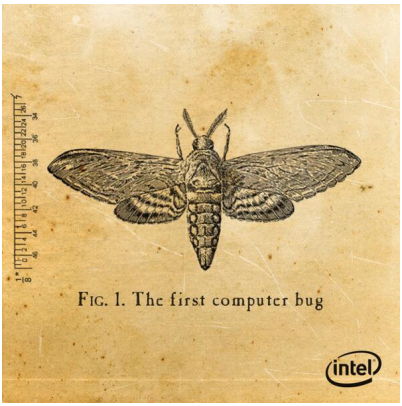
Se il tuo compagno riesce davvero ad arrivare all'uscita seguendo solo le tue istruzioni, significa che hai scritto un buon programma!

Se invece rimane bloccato contro un muro o non sa cosa fare… allora c'è un problema, quello che in gergo si chiama bug.

Perché si chiama bug?

Il termine "bug" ha una storia curiosa che risale al 1947.
Si racconta che un gruppo di tecnici che lavorava al computer Mark II dell'Università di Harvard trovò un vero insetto — una falena — incastrato in un componente, che causava un malfunzionamento!

Grace Hopper, una pioniera dell'informatica che faceva parte del team, documentò l'episodio incollando la falena sul registro di bordo e scrivendo:
"First actual case of bug being found."Da quel momento, la parola "bug" è diventata il termine più usato per indicare errori o problemi nei programmi.



Perché si chiama bug?

Anche se il termine era già usato in ingegneria per indicare difetti o guasti, questo episodio è entrato nella storia dell'informatica e ha contribuito a renderlo famoso in tutto il mondo.

ALGORITMI: LE RICETTE DELLA TECNOLOGIA

Un algoritmo è semplicemente una lista di istruzioni, scritte in un ordine preciso, che serve per raggiungere un risultato.Non è qualcosa di misterioso o complicato: è un procedimento, un metodo, un "come si fa".

Per capirlo meglio, pensa a una ricetta di cucina:

1. Metti la farina
2. Aggiungi l'acqua
3. Mescola
4. Inforna

Se segui tutti i passaggi correttamente, il dolce verrà!
Ma se sbagli l'ordine… addio torta!
Prima la metti nel forno e poi la mescoli? Impossibile.
La stessa cosa succede quando programmi un computer:

L'ordine delle istruzioni è fondamentale.

Un algoritmo deve essere chiaro, ordinato e senza confusione, perché la macchina eseguirà esattamente ciò che hai scritto, né più né meno.

PERCHÉ PROGRAMMARE È COSÌ IMPORTANTE?
Programmare significa prendere un'idea e trasformarla in qualcosa di reale.

È un superpotere moderno: ti permette di far fare alle macchine tutto ciò che vuoi, purché tu glielo sappia spiegare.

Con il codice puoi:

- Accendere un LED con un semplice comando
- Controllare un robot, facendolo camminare, girare, evitare ostacoli
- Leggere un sensore, per sapere la temperatura, la luce, il movimento
- Creare videogiochi, dai più semplici ai più complessi
- Costruire sistemi intelligenti, come allarmi, automazioni o piccoli assistenti

E non solo!

Grazie alla programmazione puoi:

- progettare strumenti utili
- dare vita ai tuoi progetti tecnologici
- imparare come ragionano le macchine
- sviluppare un metodo logico e creativo

Programmare non è solo "scrivere codice":

è trasformare la tua immaginazione in qualcosa che funziona davvero.

HARDWARE E SOFTWARE: COSA CAMBIA?

Adesso che hai capito cos'è la programmazione, possiamo fare un passo avanti verso il nostro obiettivo:
imparare a usare Arduino nel

modogiusto

Per farlo, dobbiamo distinguere due "mondi" che lavorano sempre insieme.

Sono come due metà della stessa mela:

Hardware = il corpo

Software = la mente

Senza uno dei due… nulla può funzionare davvero!

misura, emette suoni. Esempi di hardware:

HARDWARE: IL CORPO DEL SISTEMA

L'hardware è tutto ciò che puoi toccare fisicamente.

È fatto di plastica, metallo, cavi, componenti elettronici.

È la parte che reagisce al mondo esterno, che si muove, si accende, misura, emette suoni.

Esempi di hardware:

- *La scheda Arduino Nano R4*
- *un LED*
- *un pulsante*
- *una batteria*
- *un sensore di temperatura*
- *i fili jumper*



Ma attenzione:

l'hardware da solo non fa assolutamente nulla.

SOFTWARE: LA MENTE CHE COMANDA

Il software è la parte invisibile ma intelligente. È ciò che decide cosa deve fare l'hardware.

Esempi di software che usi ogni giorno:

- l'app della calcolatrice
- YouTube
- un videogioco
- un messaggio sul telefono
- e presto… il codice che scriverai nell'Arduino IDE.

Il software è formato da istruzioni, scritte in un linguaggio che la macchina può comprendere.
Senza software:

- Arduino non sa quali pin accendere
- un sensore non sa quando inviare dati
- un robot non sa muoversi

E allo stesso tempo…
senza hardware, il software non può fare nulla.

Non puoi far accendere un LED se il LED non esiste!

PERCHÉ LAVORANO INSIEME?

Immagina una lampada intelligente.

Serve:

- hardware → la lampada, il LED, i cavi
- software → il programma che dice quando deve accendersi o spegnersi

Se uno dei due manca, la lampada non funziona.

La stessa cosa succede in Arduino: tu scrivi il programma (software) e la scheda esegue ciò che hai scritto (hardware)

IL MICROCONTROLLORE: IL CERVELLO DI ARDUINO

Dentro la tua Arduino Nano R4 c'è un piccolo componente nero molto importante:

il **microcontrollore,** cioè il "cervello" della scheda.

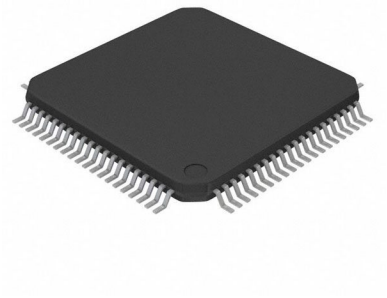
È molto più semplice di un computer, ma anche più veloce in alcune cose: esegue miliardi di operazioni al secondo ed è progettato per controllare dispositivi elettronici.

Un microcontrollore può:

- leggere ciò che arriva dai sensori
- accendere e spegnere LED
- pilotare motori
- prendere decisioni con if
- ripetere comandi molto velocemente
- comunicare con il computer tramite USB
- memorizzare il tuo programma al suo interno

Quando carichi uno sketch, è proprio il microcontrollore che lo esegue, istruzione dopo istruzione.

È grazie a lui se Arduino "vive".



FUNZIONI DISPONIBILI

Guarda il tuo multimetro: ha un selettore rotativo al centro e tanti numeri e simboli tutti intorno. Questa manopola ti permette di selezionare cosa vuoi che il multimetro misuri.

Puoi girarla in qualsiasi direzione. Se la metti esattamente al centro, di solito punterà verso la scritta OFF.

In questa posizione, il multimetro è spento.

È super importante ricordarsi di rimettere sempre la manopola su OFF quando hai finito di usarlo, così la batteria del multimetro non si scarica inutilmente!

NOTE

IL COLLEGAMENTO TRA TE E ARDUINO

Quando scriveremo i primi programmi, succederà questo:

- Tu scrivi il codice nell'Arduino IDE (software).
- Premi Carica.
- Il compilatore controlla che il codice sia corretto.
- Il programma viene inviato ad Arduino.
- Il microcontrollore lo memorizza.
- L'hardware risponde ai tuoi comandi.
- In poche parole:

Tu dai le istruzioni, Arduino obbedisce.

Che cos'è Arduino?

Al contrario di quello che si pensa, Arduino non è solo una scheda elettronica.

È molto di più: è una piattaforma completa che permette a chiunque , anche ai principianti, di creare progetti elettronici e interattivi.
Arduino è formato principalmente da tre elementi fondamentali, che lavorano insieme come un'unica squadra:



La scheda elettronica da programmare

È la parte fisica, quella che puoi toccare.
La più famosa è Arduino UNO, ma ne esistono moltissime varianti, ognuna con caratteristiche diverse.

Nel nostro percorso useremo una Arduino Nano R4, una scheda compatta e moderna, perfetta per iniziare a programmare e per costruire prototipi.



La scheda contiene:

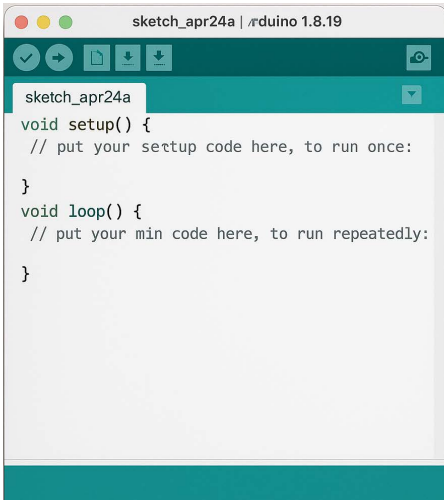
- un microcontrollore (il "cervello"),
- i pin di input e output,
- il collegamento USB,
- alcuni componenti integrati, come il LED sulla scheda.

L'ambiente di sviluppo (Arduino IDE)

Per far funzionare Arduino serve un posto in cui scrivere il codice, modificarlo e inviarlo alla scheda.
Questo posto è chiamato IDE, cioè Integrated Development Environment.
Arduino mette a disposizione due IDE diversi:

IDE 2 Desktop

È un programma da installare sul tuo computer.
È stabile, veloce e molto usato da chi lavora a progetti complessi.



Web IDE

Funziona direttamente dal browser, senza installare nulla.
Basta un account Arduino e puoi programmare la scheda anche da un'altra postazione.
In entrambi i casi, l'IDE ti permette di:

- scrivere il codice,
- verificare se ci sono errori,
- caricare il programma sulla scheda,
- usare esempi già pronti,
- gestire librerie aggiuntive.

La community: il vero cuore di Arduino

Arduino è una piattaforma Open Source.
Significa che:

- i progetti,
- la documentazione,
- i file delle schede,
- l'ambiente di sviluppo

sono pubblici e accessibili a tutti.

Nessun brevetto segreto, nessun blocco artificiale: chiunque può studiare, migliorare e condividere.

Questa scelta ha creato nel tempo una community gigantesca di appassionati, studenti, insegnanti, inventori, sviluppatori professionisti.

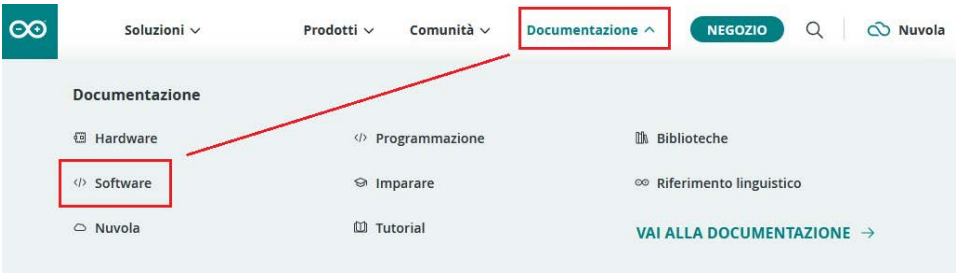
Ogni giorno la community contribuisce a migliorare Arduino:

- crea librerie nuove,
- risolve problemi,
- pubblica tutorial,
- condivide progetti,
- risponde alle domande dei principianti.

È proprio grazie a questa comunità che Arduino è diventato lo strumento educativo n°1 al mondo per imparare la programmazione e l'elettronica.

Come installare Arduino IDE

- Aprire un browser qualsiasi sul PC e inserire come indirizzo www.arduino.cc
- Nella homepage cliccare su documentazione poi su software
- Dopo aver cliccato sarà possibile scegliere
- per quale sistema operativo effettuare il download. Scegli quello giusto per il computer che stai utilizzando nella lista DOWNLOADS OPTIONS.
- Una volta finito il download lancia il programma Arduino IDE
- Appena avviato per la prima volta Arduino IDE installa dei pacchetti necessari al suo funzionamento in maniera autonoma!



Il nostro primo programma Arduino

Adesso scriveremo e capiremo insieme il nostro primissimo programma su Arduino!
Questo programma si chiama Blink, che in inglese significa fare l'occhiolino.
Ed è proprio quello che faremo fare a un LED:
accendersi e spegnersi da solo.

PASSO 1 — Collegare Arduino al computer



Ti serve:

- la tua Arduino Nano R4
- un cavo USB-C
- un computer con l'Arduino IDE

Procedi così:



1. Collega il cavo USB alla scheda Arduino.
2. Collega l'altra estremità al computer.
3. Dovresti vedere accendersi un LED sulla scheda: significa che ha ricevuto corrente.

Se non la selezioni, Arduino non saprà dove inviare il programma.

PASSO 3 — Aprire l'esempio Blink

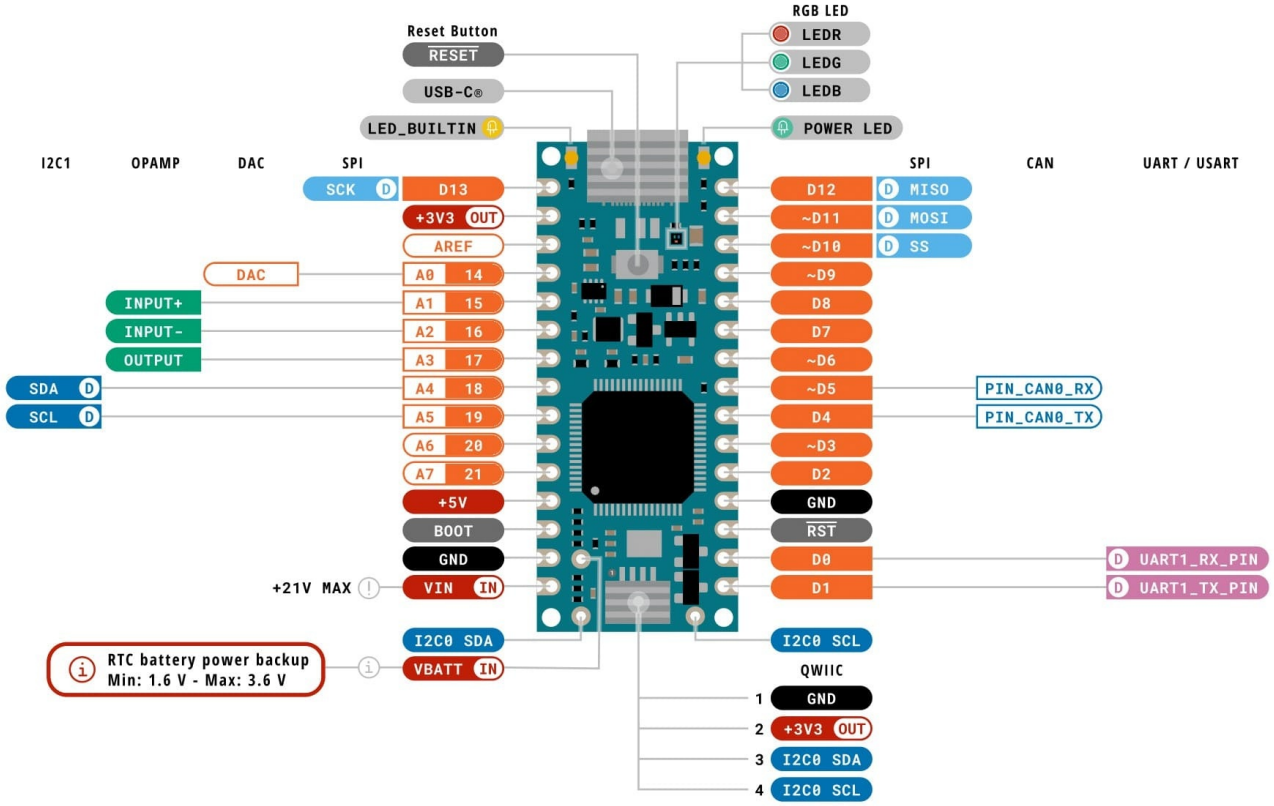


L'esempio Blink è già dentro l'IDE, pronto per essere usato:

Vai su File → Esempi → 01.Basics → Blink.
Si aprirà una nuova finestra con il programma già scritto.

Questo è il codice più famoso del mondo Arduino.

NOTE



Legend:	Digital	SPI	Internal Component
Power	Analog	UART/USART	Other Pins
Ground	I2C	Default	OPAMP

ARDUINO

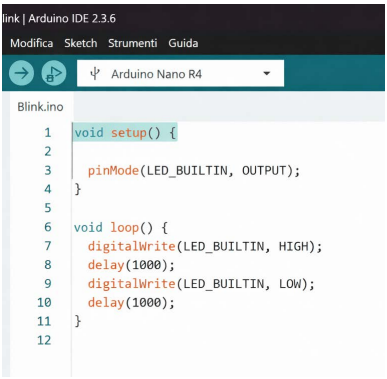
Nano R4
SKU code: ABX00142
Pinout
Last update: 11 Jul, 2025

Come è fatto un programma Arduino?

Analizziamo il programma più famoso e semplice di Arduino:

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```

Un programma Arduino ha quasi sempre 2 parti importanti:



Un programma Arduino ha quasi sempre 2 parti importanti:

setup()
È la preparazione.
Viene eseguita una sola volta, quando Arduino si accende o si resetta.
Qui diciamo ad Arduino come deve essere usato l'hardware (pin, sensori, ecc.).

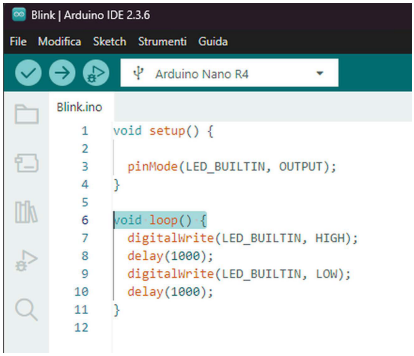
loop()
È il ciclo che si ripete.
Il codice dentro loop() viene eseguito all'infinito, in tondo: quando arriva alla fine, ricomincia dall'inizio.

Puoi immaginare:
setup() = "sistemiamo le cose prima di iniziare a giocare"
loop() = "il gioco vero e proprio che ripetiamo in continuazione"

Spiegazione parola per parola

RIGA 1:
void setup() {

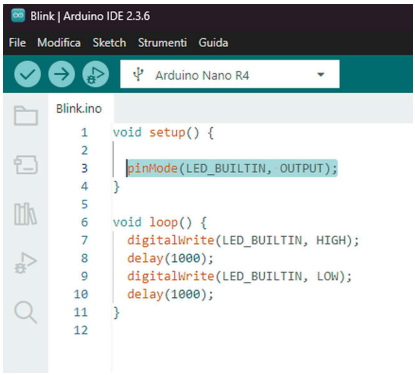
void . In inglese significa “vuoto”.
Qui vuol dire: questa funzione non restituisce niente (non ti dà un numero o un risultato da usare dopo).
È solo un gruppo di comandi da eseguire.



setup

Significa preparazione.
È il nome speciale della funzione che Arduino cerca sempre all’inizio.

( ) *(Parentesi aperta e chiusa)*
Le parentesi tonde.
Servono per dire se la funzione ha bisogno di qualche dato in ingresso.
Qui sono vuote → setup non ha bisogno di informazioni in ingresso.



RIGA 2:
" pinMode(LED_BUILTIN, OUTPUT);
} "

pinMode
È una funzione/comando.
“Pin” = piedino di Arduino (uno dei buchini numerati).

“Mode” = modalità.

Quindi pinMode serve per dire: come useremo quel pin? Come ingresso o come uscita?

" (" *(Parentesi chiusa)*

Inizia la lista delle informazioni che la funzione pinMode deve usare.

LED_BUILTIN

LED = la lucina.

BUILTIN = “incorporato”, cioè già montato sulla scheda.

È come se dicessimo: “usa la lucina interna della scheda”.

" , " *(virgola)*
Separa il primo dato (quale pin) dal secondo dato (come lo useremo).

"OUTPUT" *Significa uscita.*

Vuol dire: “da questo pin usciranno segnali”, cioè Arduino userà il pin per mandare corrente (accendere cose, come il LED).

") " *(Parentesi chiusa)*

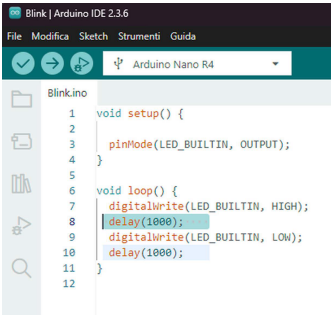
Chiude la lista di informazioni della funzione.

" ; " (punto e virgola)

È come il punto alla fine di una frase. Dice ad Arduino: “questa istruzione è finita, passa alla prossima”.

" } " *(Parentesi graffa chiusa.)*

Vuol dire: “qui finisce il blocco di comandi di setup”.
Quindi setup ha solo una riga di comandi, quella con pinMode.



RIGA 3:

" void loop() {

" void "

È come prima: funzione che non restituisce un valore.

" loop "

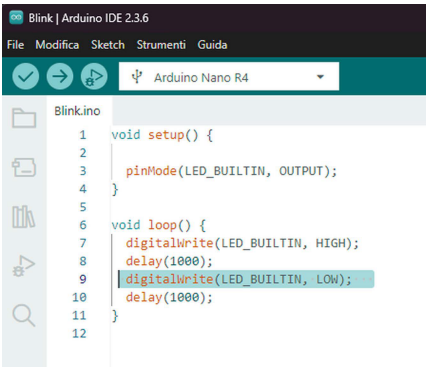
In inglese significa “giro” / “ciclo”. È il nome speciale della funzione che Arduino ripete all’infinito.

" () " *(Parentesi aperta e chiusa)*

Come prima: parentesi tonde, qui vuote perché la funzione non ha bisogno di dati in ingresso.

" { " *(Parentesi graffa aperta)*

Inizio del blocco di comandi che appartiene a loop.



RIGA 7:

digitalWrite(LED_BUILTIN, HIGH);

digitalWrite

"digital" = che può valere solo 0 o 1, spento o acceso.

"write" = scrivere.

Insieme: scrivere un valore digitale su un pin, cioè dire ad Arduino se quel pin deve essere spento (0) o acceso (1).

" (" *(Parentesi chiusa)*

Inizio dei dati da passare alla funzione.

LED_BUILTIN

Di nuovo, il LED interno alla scheda.

" , " (virgola)

Separatore tra primo dato (il pin) e secondo dato (il valore).

HIGH *"In inglese "alto".*

Per Arduino vuol dire: accendi, manda corrente, metti quel pin a 1.

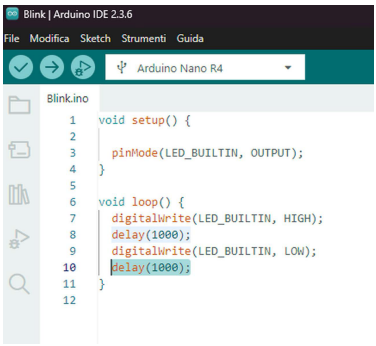
") " *(Chiusa parentesi)*

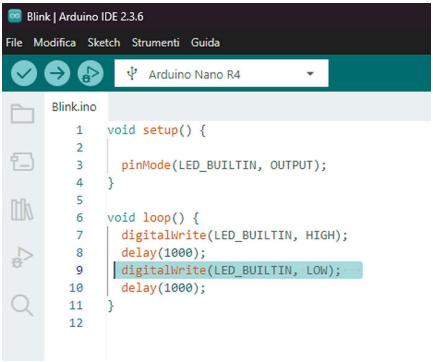
Fine dei dati per digitalWrite.

" ; " *(Punto e virgola)*

Fine dell'istruzione.

Spiegazione logica della scritta:
"Accendi il LED interno della scheda".





RIGA 8/10: delay(1000);

delay

Significa “pausa / ritardo”.

Dice ad Arduino di aspettare un po’ prima di fare " il prossimo comando.

" (" *(aperta parentesi)*

Inizio del numero che indica quanto tempo aspettare.

1000

Numero di millisecondi.

1000 millisecondi = 1 secondo.

Quindi: “aspetta 1 secondo”.

") " *(Chiusa parentesi)*

Fine del tempo specificato.

" ; " *(punto e virgola)*

Fine dell’istruzione.

Spiegazione logica della scritta:

“Ora aspetta 1 secondo senza fare altro.”

RIGA 9: digitalwrite(LED_BULDING, LOW);

È come la riga 5, ma:

- LOW
  - In inglese “*basso”.*
  - Vuol dire: spegni, niente corrente, metti quel pin a 0.

Spiegazione logica della scritta:

“Spegni il LED interno della scheda.”

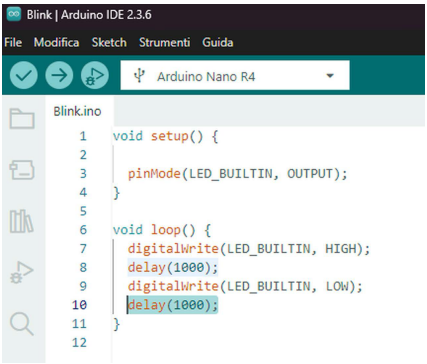
RIGA8/10: delay(1000);

È identica alla riga 7:

di nuovo aspetta 1 secondo.

Spiegazione logica della scritta:
"Ora aspetta 1 secondo senza fare altro."

Cosa succede davvero passo passo



- All'accensione Arduino esegue **setup():**
- imposta il LED interno come uscita **pinMode(LED_BUILTIN, OUTPUT);**
- }

- Poi entra in **loop()** e fa sempre gli stessi passi:
- **digitalWrite(LED_BUILTIN, HIGH);** → accende il LED.
- **delay(1000);** → aspetta 1 secondo.
- **digitalWrite(LED_BUILTIN, LOW);** → spegne il LED.
- **delay(1000);** → aspetta 1 secondo.
- Torna all'inizio di loop() e ricomincia. }
- **}**
-

Risultato: la lucina sulla scheda Arduino lampeggia: 1 secondo accesa, 1 secondo spenta, all'infinito

Promemoria dei termini

NOTE DI ELAB – Da tenere a portata di mano

**GLOSSARIO RAPIDO –
Capitolo 5**

1. **Sketch / Programma** — L'insieme di istruzioni che Arduino esegue.
2. **Istruzione (riga di codice)** — Un comando singolo che Arduino deve fare.
3. **setup()** — La fase di preparazione: si esegue una sola volta all'avvio o al reset.
4. **loop()** — Il ciclo: si ripete all'infinito.
5. **void** — Significa "vuoto": la funzione non restituisce un risultato, esegue solo comandi.
6. **Funzione** — Un gruppo di comandi con un nome (es. setup, loop, delay…).
7. **( ) (parentesi tonde)** — Contengono i dati usati da una funzione (es. delay(1000)).
8. **{ } (parentesi graffe)** — Indicano inizio e fine di un blocco di comandi.
9. **pinMode(pin, modalità)** — Dice ad Arduino come usare un pin.
10. **OUTPUT** — Il pin è un'uscita: Arduino può accendere o spegnere qualcosa.
11. **LED_BUILTIN** — La lucina già presente sulla scheda (LED interno).
12. **digitalWrite(pin, valore)** — Scrive su un pin digitale: acceso/spento.
13. **HIGH** — "Alto": acceso, corrente presente (1).
14. **LOW** — "Basso": spento, niente corrente (0).
15. **delay(tempo)** — Fa una pausa prima del comando successivo.
16. **Millisecondi (ms)** — 1000 ms = 1 secondo.
17. **, (virgola)** — Separa i dati dentro una funzione.
18. **; (punto e virgola)** — Fine di un'istruzione (come il punto a fine frase).
19. **Compilatore** — Controlla se il codice è scritto bene; se trova errori non carica il programma.
20. **Reset** — Riavvio della scheda: setup() riparte da capo.

NOTE

I PIN DIGITALI: LE DITA DI ARDUINO

«Bentornati, giovani esploratori!

Fino a ora Arduino ha acceso e spento una lucina sulla sua pancia… ma oggi scopriremo una cosa incredibile: Arduino ha tante dita e con ognuna può accendere e spegnere qualcosa nel mondo reale!

Obiettivo del capitolo

In questo capitolo imparerai:

- cosa sono i pin digitali
- cosa significano HIGH e LOW
- perché i pin funzionano come interruttori
- come Arduino accende e spegne componenti esterni
- a creare giochi di luce sempre più interessanti

Alla fine del capitolo, Arduino non controllerà più solo una lucina sulla scheda… ma componenti veri collegati fuori, come un vero cervello elettronico.

I pin digitali: cosa sono davvero?

Guarda la tua Arduino Nano R4.
Ai lati della scheda vedi tanti buchini numerati:
D2, D3, D4… D13.
Quelli che iniziano con la lettera D si chiamano: **pin digitali**
Un pin digitale è un punto da cui Arduino può:

- far uscire corrente
- oppure non far uscire nulla

Non esistono vie di mezzo, solo due possibilità. Un pin digitale può avere solo due stati:

- ACCESO
- SPENTO

Proprio come:

- un interruttore della luce
- un pulsante ON / OFF
- una porta aperta o chiusa

nota ELAB

- Un pin digitale non è complicato: o fa passare corrente, oppure no.

HIGH e LOW: il linguaggio segreto dei pin
Arduino non usa le parole “acceso” e “spento”.
Usa due parole speciali in inglese:

HIGH , significa alto ,vuol dire:

- accendi
- manda corrente
- sì

LOW, significa basso , vuol dire:

- spegni
- niente corrente
- no

Esempio pratico

- LED acceso → pin in HIGH
- LED spento → pin in LOW

Arduino parla così con l’elettronica.

I pin digitali come interruttori Ora arriva una delle idee più importanti di tutto Arduino.

· Un pin digitale è come un interruttore elettronico.
La differenza è che:

- non lo premi con il dito
- lo comandi con il codice

Quando scrivi un programma, è come se dicessi:

- “Accendi questo interruttore”
- “Spegni quest’altro”

OUTPUT: quando Arduino comanda

Quando vogliamo che Arduino accenda o spenga qualcosa, usiamo il pin come:
OUTPUT *(uscita)*

Per questo, prima di usarlo, dobbiamo sempre dire:

pinMode(numero_pin, OUTPUT);

È come dire ad Arduino:
“Questo pin lo userai per dare ordini.”

Le due funzioni magiche

Per controllare i pin digitali useremo sempre due funzioni.

pinMode() – preparo il pin
Serve per dire ad Arduino come useremo un pin.
Esempio:
pinMode(13, OUTPUT);

Significa:
“Il pin 13 sarà un’uscita.”

digitalWrite() – accendo o spengo
Esempio:

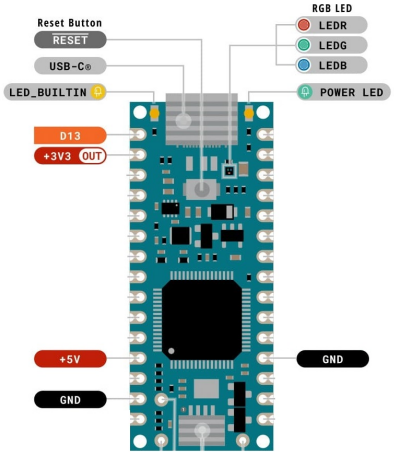
digitalWrite(13, HIGH);
Significa: “Accendi il pin 13.”

digitalWrite(13, LOW);
Significa: “Spegni il pin 13.”

Premessa:
(alimentazione breadboard)

Ragazzi, nei Volumi 1 e 2 abbiamo quasi sempre alimentato la breadboard usando una batteria da 9V (con il suo cavetto/clip), quindi la corrente arrivava alla breadboard direttamente dalla batteria.

Da adesso, con Arduino Nano R4, cambiamo metodo: non usiamo più la 9V per alimentare la breadboard, perché l'alimentazione può arrivare direttamente dalla scheda Arduino collegando il cavo usbC al PC.

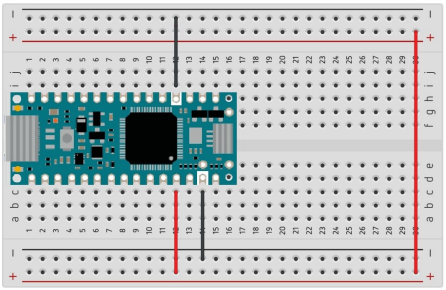


Come funziona ora:

La breadboard prende la corrente dalla scheda Arduino.

Il positivo (+) lo prendiamo dal pin 5V dell'Arduino (quello indicato proprio come 5V).

Il negativo (–) lo prendiamo da uno dei pin **GND** (massa) indicati sulla scheda.



ESPERIMENTO 1:

- Materiale necessario
- 1 Arduino Nano R4
- 1 breadboard (400 punti)
- 1 LED (rosso o verde)
- 1 resistenza 470 Ω
- 1 cavetto jumper
- Cavo USB-C

Posizioniamo Arduino Nano sulla breadboard

Prendi la breadboard.

Inserisci la Arduino Nano in modo che: i pin di sinistra stiano su una riga "**a**", i pin di destra sulla riga "**g**"

La Nano rimane a cavalo della rigache separa le breadbord, così ogni pin resta separato. in questo modo puoi usare facilmente i pin D2, D3, D4… D13 e i pin GND, 5v

Spostiamo il LED sulla breadboard

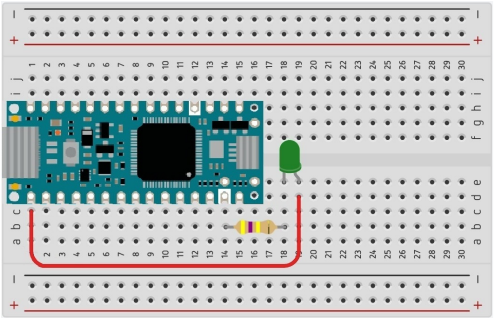
Useremo un LED verde per riprodurre lo stesso effetto del LED_BUILTIN, ma questa volta fuori dalla scheda.

- Lunga = anodo (+)
- Corta = catodo (–)

Colleghiamo la resistenza

La resistenza deve andare in serie al led .Quindi metti un capo della resistenza nella stessa fila della gamba corta del LED.

L'altro capo lo mettiamo nella colonna del gnd della nostra NANO



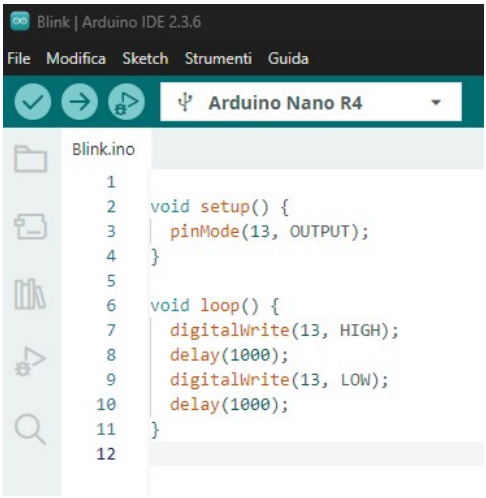
Collegamenti elettrici con Arduino Nano

D13 → gamba lunga del LED (anodo)

(useremo D13 perché è quello che controlla il LED interno, così il codice resta identico)

GND → resistenza → gamba corta del LED (catodo)

Codice da caricare (lo stesso del capitolo 5 l'unica differenza che invece di usare il termine "LED_BUILTIN" lo sostituiamo con "13 "



Nota: su Arduino Nano è meglio usare il numero del pin (13) invece di LED_BUILTIN, per essere più chiari nella pratica.

Esperimenti aggiuntivi (per approfondire)

Cambia la velocità del lampeggio

Modifica il delay:

- 200 ms → lampeggio veloce
- 2000 ms → lampeggio lento

ESPERIMENTO 2:

Cambia il numero di pin sia sul codice che sulla Breadbord

Modifica il pin:

Prova a usare:

pinMode(5, OUTPUT);

digitalWrite(5, HIGH);

digitalWrite(5, LOW);

ESPERIMENTO 3:

Fai un SOS in codice Morse

Modifica il codice aggiungi comandi:

Prova a usare:

void loop() {

digitalWrite(13, HIGH);

delay(1000);

digitalWrite(13, LOW);

delay(200);

digitalWrite(13, HIGH;

delay(200);

digitalWrite(13, LOW);

delay(2000);

}

ESPERIMENTO 4 :

PROGRAMMA LUCI LAMPEGGIANTE STILE SIRENATO 2 :

Due LED: effetto “polizia”

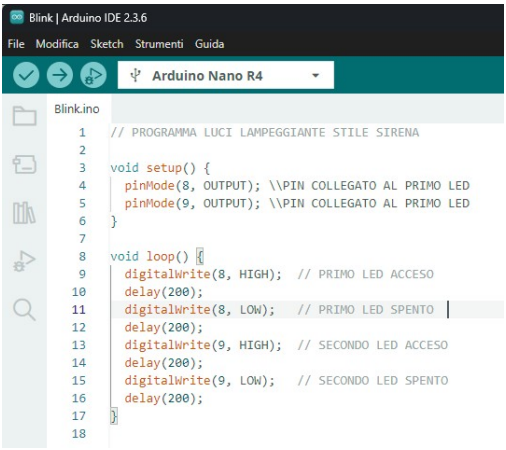
Prova a usare:
Collega:

- LED 1 → pin 8
- LED 2 → pin 9

Programma:

- uno acceso
- l’altro spento
- poi si scambiano

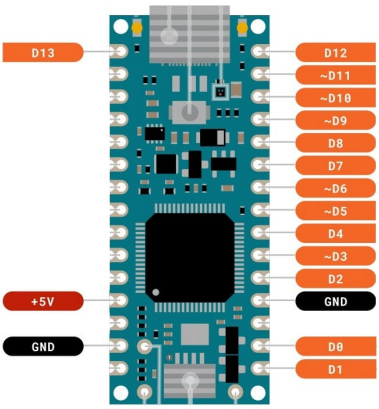
Risultato: luci lampeggianti stile sirena!



Nota di elab

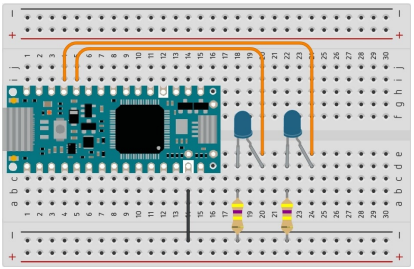
Nella figura affianco abbiamo inserito delle scritte di spiegazione per chiarire il significato e il funzionamento di ogni singola stringa/istruzione del programma.

Queste spiegazioni sono state aggiunte sotto forma di commenti, riconoscibili perché prima del testo compaiono due barre:
// esempio di nota



In questo modo possiamo scrivere tutte le annotazioni che vogliamo direttamente sulla riga interessata

(o anche su più righe), senza modificare il comportamento del codice: i commenti infatti non vengono eseguiti e quindi non interferiscono con il programma scritto nell’IDE.



Note utili

Usa i commenti per: spiegare cosa fa una riga, chiarire passaggi complessi, indicare valori importanti, o ricordare scelte fatte durante lo sviluppo.

Mantieni i commenti brevi e chiari, così il codice resta leggibile.

Se cambi il codice, aggiorna anche i commenti: devono sempre
"seguire" il programma per evitare

ESPERIMENTO 5: Il semaforo

Un semaforo è solo: un Blink

con più fasi

LED:

Verde → pin 5

Giallo → pin 6

Rosso → pin 7

Sequenza:

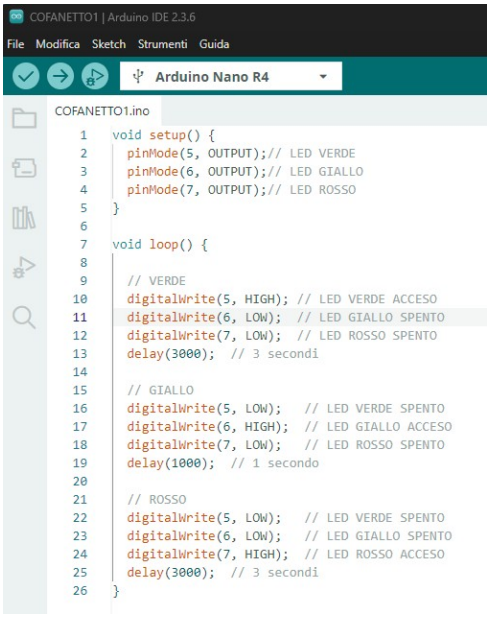
Verde (3 s)

Giallo (1 s)

rosso (3 s)

Ripeti

Prova a scriverlo da solo poi controlli le stringe con questo esempio.



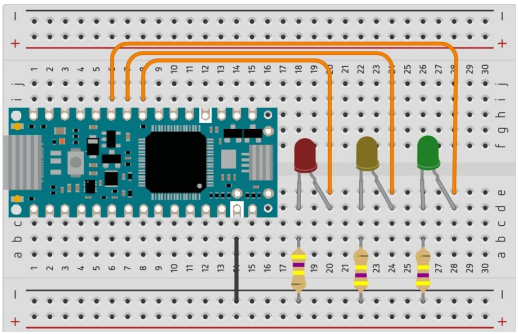

```ino
void setup() {
  pinMode(5, OUTPUT);// LED VERDE
  pinMode(6, OUTPUT);// LED GIALLO
  pinMode(7, OUTPUT);// LED ROSSO
}

void loop() {

  // VERDE
  digitalWrite(5, HIGH); // LED VERDE ACCESO
  digitalWrite(6, LOW);  // LED GIALLO SPENTO
  digitalWrite(7, LOW);  // LED ROSSO SPENTO
  delay(3000);  // 3 secondi

  // GIALLO
  digitalWrite(5, LOW);   // LED VERDE SPENTO
  digitalWrite(6, HIGH);  // LED GIALLO ACCESO
  digitalWrite(7, LOW);   // LED ROSSO SPENTO
  delay(1000);  // 1 secondo

  // ROSSO
  digitalWrite(5, LOW);   // LED VERDE SPENTO
  digitalWrite(6, LOW);   // LED GIALLO SPENTO
  digitalWrite(7, HIGH);  // LED ROSSO ACCESO
  delay(3000);  // 3 secondi
}
```



Hai appena realizzato:

- il primo circuito fisico con Arduino Nano,
- il primo programma che controlla un componente esterno,
- il collegamento tra software (codice) e hardware (LED + breadboard).

Errori tipici (diventa un detective)

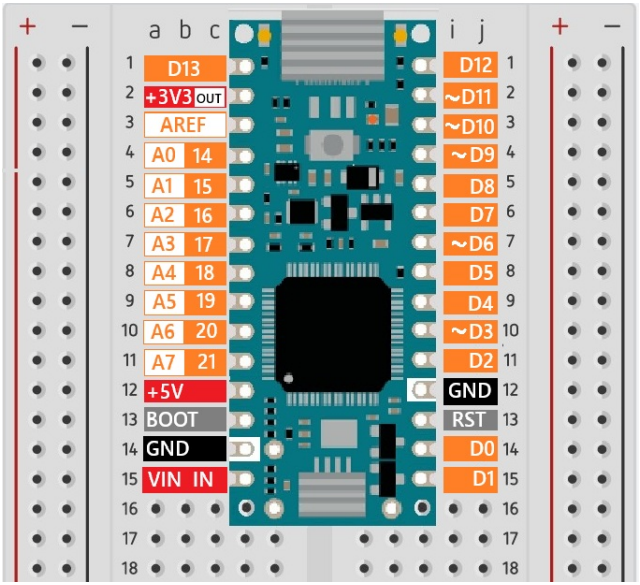
Se il LED non si accende, controlla:

Checklist:

- LED girato nel verso giusto?
- Resistenza collegata bene?
- GND presente?
- Numero del pin corretto sia sulla breadbord che nel codice?

Nota ELAB

Un buon programmatore controlla sempre anche i fili.



NOTE DI ELAB - Promemoria dei termini (Capitolo 6)

TERMINE	
Pin digitali (D2, D3... D13)	Piedini che possono stare solo in due
HIGH	Stato 'acceso': il pin vale 1 e manda corrente.
LOW	Stato 'spento': il pin vale 0 e non manda corrente.
Interruttore elettronico	Un pin digitale e' come un interruttore: lo accendi o lo spegni con il codice.
OUTPUT	Modalita' per comandare qualcosa: Arduino usa il pin per far uscire un segnale.
pinMode(pin, OUTPUT)	Dice ad Arduino: 'questo pin lo usero' come uscita'.
digitalWrite(pin, HIGH/LOW)	Accende o spegne un pin digitale (e quindi un LED collegato
LED (anodo e catodo)	Gamba lunga = anodo (+). Gamba corta = catodo (-).
Resistenza in serie (220-470 ohm)	Limita la corrente per proteggere il LED: va messa in serie al LED.
GND	Massa/zero volt: e' il ritorno della corrente (il 'negativo').
5V	Alimentazione positiva dalla scheda (il 'positivo').
D13 e LED_BUILTIN	D13 e' il pin collegato alla lucina interna. Per un LED esterno puoi usare anche il numero 13.
delay(ms)	Pausa: ferma Arduino per un tempo in millisecondi (1000 ms = 1 secondo).
Millisecondo (ms)	Pausa: ferma Arduino per un tempo in millisecondi (1000 ms = 1 secondo). Unita' di tempo: 1000 ms = 1 secondo.
Commenti //	Note per gli umani: Arduino le ignora. Servono per ricordare e spiegare il codice.
Sequenza / fasi	Una sequenza e' una serie di accensioni con tempi diversi (es. semaforo: verde, giallo, rosso).
Debug (checklist)	Se non funziona: LED nel verso giusto? resistenza presente? GND collegato? pin giusto nel codice?

NOTE

I PIN DI INPUT: ARDUINO ASCOLTA E DECIDE

Giovani esploratori, fino a questo momento Arduino è stato un ottimo esecutore.
Ha acceso luci, le ha spente, ha fatto lampeggiare LED come un vero mago dell'elettronica.

Ma c'è una domanda importante da farsi:

Arduino può reagire a quello che facciamo noi?

La risposta è sì.
E oggi Arduino impara una nuova abilità: ascoltare.

7.1 INPUT e OUTPUT: Arduino parla e Arduino ascolta

Nel Capitolo 6 abbiamo usato i pin digitali come OUTPUT (uscita). Cosa vuol dire?

Arduino mette un pin in **HIGH** → esce "energia" → LED si accende
Arduino mette un pin in **LOW** → non esce energia → LED si spegne

Quindi:

OUTPUT = Arduino comanda



Ora invece vogliamo che Arduino riceva un'informazione dall'esterno.
Per esempio: "Qualcuno sta premendo un pulsante?"

INPUT = Arduino ascolta



È come dire:
OUTPUT → Arduino usa la bocca
INPUT → Arduino usa le orecchie

7.2 Il pulsante: come funziona davvero

Un pulsante è un componente semplice ma geniale.

Quando NON premi

Il circuito è aperto → non passa corrente.

Quando premi

Il circuito si chiude → passa corrente.

Quindi il pulsante manda ad Arduino un messaggio che ha solo due possibilità:

- sì / no
- acceso / spento
- HIGH / LOW

Come ti ricorderai nel volume uno di Elab abbiamo visto come collegare il pulsante, ricordati il pulsante va sempre inserito a cavallo della riga centrale della breadbord.

7.3 INPUT_PULLUP: la scorciatoia più utile per iniziare

Ora arriva un trucco utilissimo.
In teoria, per far funzionare bene un pulsante, servirebbe una resistenza "di aiuto".
Ma Arduino ce l'ha già dentro!
Quindi invece di scrivere:
pinMode(2, INPUT);
noi useremo:
pinMode(2, INPUT_PULLUP);

Questa modalità: rende la lettura stabile
riduce i collegamenti evita falsi valori "strani"
Ma c'è un effetto curioso

Con INPUT_PULLUP:

pulsante NON premuto → HIGH

pulsante premuto → LOW

Sì: è "al contrario".

Ma è normale: basta ricordarselo.

Perché non basta scrivere INPUT?
Quando scrivi: pinMode(2, INPUT);

Arduino dice:
"Ok, il pin 2 adesso deve ascoltare."
MA… c'è un problema grosso:
Un pin in INPUT è come un orecchio super sensibile.Se non gli dai un riferimento, il pin resta "in aria", cioè non è collegato né a 5V né a GND.
E un pin "in aria" può:
leggere HIGH anche se non tocchi niente
leggere LOW a caso cambiare valore solo perché muovi un filo o avvicini la mano
Questo si chiama **pin fluttuante** (floating).
È come chiedere a qualcuno:
"Hai sentito un rumore?"
ma stai in una stanza piena di eco:
sentirà "qualcosa" anche se non c'è.

Quindi a cosa serve INPUT_PULLUP?
Quando scrivi:
pinMode(2, INPUT_PULLUP);
Arduino fa due cose insieme: mette il pin in INPUT (ascolta) attiva una resistenza interna che lo "tira su" verso HIGH

Cosa significa "tira su"?

Significa che Arduino dice al pin:
"Se nessuno ti sta dicendo nulla, tu stai su **HIGH,** in modo stabile."

Questa resistenza interna si chiama **pull-up**.
Così il pin non resta più "in aria"

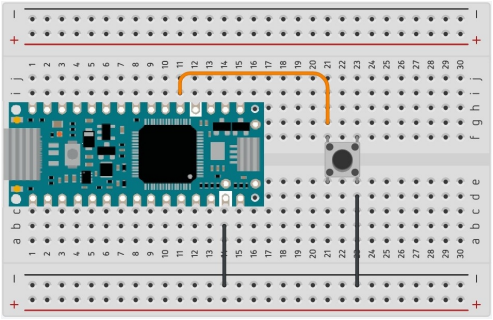
Con INPUT_PULLUP succede questo:
Quando NON premi il pulsante
il pulsante è aperto (non collega niente)
il pin NON resta nel vuoto
la resistenza interna lo tiene su → **HIGH** stabile

Quando PREMI il pulsante
il pulsante collega il pin a GND
quindi il pin va a 0V → **LOW**

Ecco perché **"premuto = LOW"** (sembra al contrario ma è normale).

ESERCIZIO 7.3 – Collegamento del pulsante con INPUT_PULLUP (super semplice)

Obiettivo: collegare il pulsante usando solo 1 filo + GND. Scegliamo il pin di lettura:
useremo D2. Collega un cavetto da D2 a un lato del pulsante.
Collega l'altro lato del pulsante a GND.Fatto.



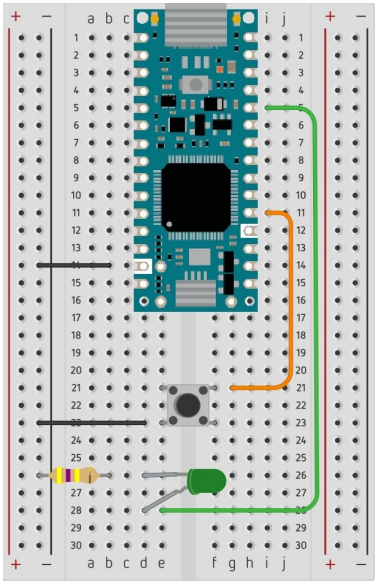
In questo modo:

se premi → D2 si collega a GND → Arduino legge LOW

se non premi → Arduino legge HIGH (grazie al pullup interno)

7.4 Arduino legge "premuto O non premuto" digitalRead():

Ora Arduino deve leggere lo stato del pulsante. Nel capitolo precedente per accendere i led abbiamo usato digitalWrite(2, ...) quindi diciamo noi se scceso o spento, ora dobbiamo leggere il valore del pulsante quindi la funzione è: digitalRead(2, ...)



Questa funzione restituisce: HIGH o LOW
Ecome chiedere: il Pin 2, mi stai dicendo HIGH o LOW?

7.6 Arduino decide: if, else e perché usiamo " == "

Ora arriva la magia vera della programmazione.
Arduino non deve solo leggere: deve decidere.
Usiamo:

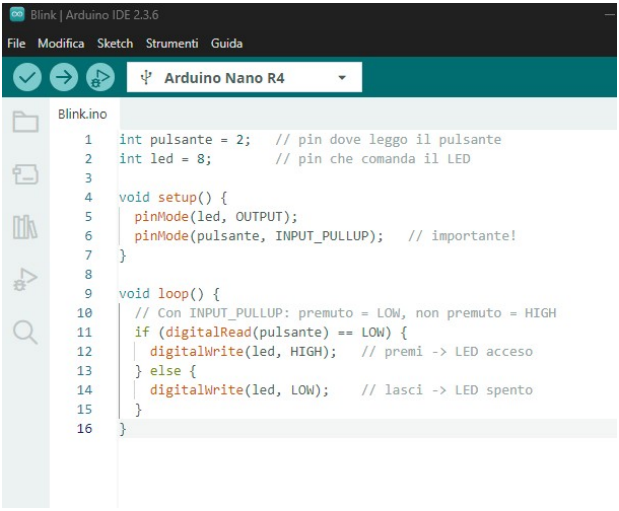
if (condizione) int pulsante = 2;
int led = 8; // fai questo
else { } // altrimenti fai quest'altro

SE succede una cosa → fai A ALTRIMENTI → fai B
Nel nostro caso:
Se il pulsante è premuto (LOW) → accendi LED ALTRIMENTI → spegni LED



"int" e " == "

Abbiamo aggiunto un altro termine al programma nella pagina successiva ti spiego tutto!



7.7 Le “scatoline” dei numeri: " int "

Quando scriviamo:

```
int pulsante = 2;
int led = 8;
```

stiamo dicendo ad Arduino:

“Mi serve una scatolina dove conservare un numero.”

int significa: numero intero (cioè un numero senza virgola: 0, 1, 2, 8, 13, 500…)

Quindi:
pulsante == è una scatolina che contiene il numero 2
led == è una scatolina che contiene il numero 8

Perché lo facciamo?

Perché invece di scrivere sempre:

```
pinMode(8, OUTPUT);
pinMode(2, INPUT_PULLUP);
digitalRead(2);
digitalWrite(8, HIGH);
```

Possiamo scrivere:

```
pinMode(led, OUTPUT);
pinMode(pulsante, INPUT_PULLUP);
digitalRead(pulsante);
digitalWrite(led, HIGH);
```

Così il programma è più chiaro, perché leggendo capisci subito:

quale pin è il pulsante
quale pin è il LED

ESERCIZIO – Cambia un numero, cambia il circuito
Sposta il filo del LED da D8 a D9

Cambia solo questa riga:
int led = 8; ==> cambialo con int led = 9;

Se funziona, hai capito cos’è una variabile:
hai cambiato un solo numero e hai cambiato il comportamento del programma.

7.7 Perché si scrive " == " (due uguali)?

In Arduino (e in quasi tutti i linguaggi):

"=" (uno uguale) significa assegna.

Cioè: "metti dentro".

Esempio:

int led = 8; vuol dire: metti il numero 8 nella scatolina chiamata led.

come dire: *"led prende valore 8".*

In Arduino (e in quasi tutti i linguaggi): "**==** " (due uguali) significa confronta

Cioè: "è uguale a...?"

Esempio:

digitalRead(pulsante) == LOW

vuol dire: "Quello che leggo dal pulsante è uguale a LOW?"

una domanda a cui Arduino risponde con:

si (vero)

no (falso)

E poi, grazie a **" if "**, decide cosa fare.

Esempio super chiaro con frasi

if (digitalRead(pulsante) == LOW) { Si legge cosa:

"SE il pulsante è premuto (cioè LOW), allora..."

Errore tipico da evitare Questo è sbagliato:

if (digitalRead(pulsante) = LOW)

Perché = non fa una domanda: prova ad "assegnare" LOW (e crea confusioneKerrori).

Quello giusto è:
if (digitalRead(pulsante) == LOW)

" = " e " == " non sono la stessa cosa

= significa assegna
(metti un valore in una variabile)

== significa confronta (sono uguali?)

Dentro if(...) si usa quasi sempre == perché stiamo facendo una domanda.

7.7 Debug guidato (se non funziona)

Se il LED non risponde:

- Il LED è girato bene? (gamba lunga verso il pin, corta verso resistenza/GND)
- La resistenza è collegata davvero in serie?
- Il pulsante attraversa la fessura centrale?
- Il pulsante è collegato tra D2 e GND?
- Nel codice:
- pulsante =
- led =
- INPUT_PULLUP scritto giusti
- confronto con == (due uguali)

7.8 Mini-progetto: Due LED, un pulsante (Arduino sceglie)

Obiettivo

se premi → LED verde acceso
se non premi → LED rosso acceso

Collegamenti

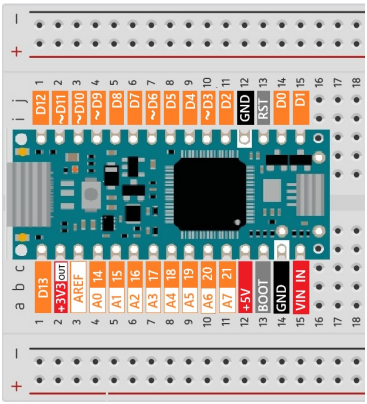
LED verde → D8
LED rosso → D9
pulsante → D2 e GND

Codice
INPUT_PULLUP scritto giusto
confronto con == (due uguali)

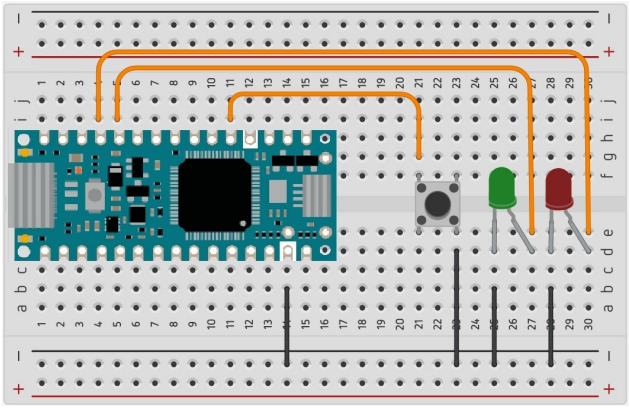
programma:




```
1  int pulsante = 2;   // Significa: useremo il pin D2 per LEGGERE il pulsante (INPUT).
2  int verde = 8;      // Significa: useremo il pin D8 per COMANDARE il LED verde (OUTPUT).
3  int rosso = 9;      // Significa: useremo il pin D9 per COMANDARE il LED rosso (OUTPUT).
4
5  void setup() {      // setup() è la fase di PREPARAZIONE: Arduino la esegue UNA SOLA VOLTA all'inizio.
6    pinMode(verde, OUTPUT);      // Diciamo ad Arduino: il pin del LED verde (D8) è un'uscita.
7    pinMode(rosso, OUTPUT);      // Diciamo ad Arduino: il pin del LED rosso (D9) è un'uscita.
8    pinMode(pulsante, INPUT_PULLUP); // Diciamo ad Arduino: il pin del pulsante (D2) è un ingresso.
9                                     // INPUT = Arduino deve ASCOLTARE.
10                                    // PULLUP = Arduino attiva una resistenza interna che tiene il pin su HIGH
11                                    //          quando non premi il pulsante (così non legge valori a caso).
12                                    // IMPORTANTE: con INPUT_PULLUP vale questa regola:
13                                    // - NON premuto = HIGH
14                                    // - PREMUTO     = LOW (perché il pulsante collega il pin a GND)
15 }
16
17 void loop() {       // loop() è il ciclo: Arduino ripete queste istruzioni ALL'INFINITO.
18
19   // Ora Arduino legge lo stato del pulsante e decide cosa fare con i due LED.
20   // digitalRead(pulsante) può valere solo HIGH oppure LOW.
21
22   if (digitalRead(pulsante) == LOW) {  // Qui facciamo una domanda:
23                                        // "Il pulsante è PREMUTO?"
24                                        // Usiamo == perché significa "è uguale a?"
25                                        // Con INPUT_PULLUP: se è premuto, il valore letto è LOW.
26     digitalWrite(verde, HIGH);   // Se il pulsante è premuto:
27                                  // accendo il LED verde (metto il pin D8 a HIGH).
28     digitalWrite(rosso, LOW);    // e nello stesso tempo spengo il LED rosso (metto il pin D9 a LOW).
29
30   } else {                       // ALTRIMENTI (cioè se il pulsante NON è premuto):
31
32     digitalWrite(verde, LOW);    // spengo il LED verde
33     digitalWrite(rosso, HIGH);   // e accendo il LED rosso
34   }
35 }
```



Scema di assemblaggio sulla breadbord:



Pull-up interno	Resistenza interna che tiene il pin su HIGH quando non premi.
Regola INPUT_PULLUP	NON premuto = HIGH • PREMUTO = LOW (perché va a GND).
digitalRead(pin)	Legge un pin digitale e restituisce HIGH oppure LOW.
digitalWrite(pin, …)	Scrive su un pin: lo mette HIGH (accende) o LOW (spegne).
if	"SE": controlla una condizione e decide se fare un'azione.
else	"ALTRIMENTI": cosa fare quando la condizione dell'if è falsa.
Condizione	Una domanda vera/falsa (es. "pulsante premuto?").
int	Numero intero: crea una "scatolina" per conservare un numero (es. pin 2, 8, 9).
Variabile	Una scatolina con un nome (es. pulsante, verde) che contiene un valore.
= (un uguale)	Assegna: mette un valore in una variabile (es. int led = 8;).
== (due uguali)	Confronta: chiede "è uguale a…?" (si usa dentro if).

NOTE

I PIN ANALOGICI: ARDUINO IMPARA A SENTIRE LE QUANTITÀ

«Giovani esploratori, nel Capitolo 7 Arduino ha imparato a fare una cosa da "cervello":ascoltare e decidere con **SE... ALLORA...**

Però ascoltava solo messaggi molto semplici:

- sì / no
- acceso / spento
- IGH / LOW

Ma il mondo reale è più ricco.

La luce non è solo "c'è / non c'è": può essere poca, media, tanta.
E il volume non è solo "muto / alto": può crescere piano piano.

Oggi Arduino impara a sentire le quantità.».

8.0 Obiettivo del capitolo

In questo capitolo imparerai:

- cos'è un segnale analogico
- cosa sono i pin A0, A1, A2...
- come usare la funzione
- analogRead()
- cosa significa leggere un valore da 0 a 1023
- come trasformare un numero in un'azione con if
- mini-progetti con LED controllati da una "manopola"

8.1 Digitale vs Analogico (spiegato bene)
I pin Digitali li abbiamo visti nel capitolo precedente, è come una risposta secca:

- SÌ / NO
- ON / OFF
- HIGH / LOW

Esempio:
pulsante premuto? sì/no
Il Pin Analogico
È come una quantità che può cambiare:
poco / medio / tanto
0… 1… 2… 3… fino a tantissimi valori

Esempio:
quanto stai girando una manopola?
quanta luce c'è?
quanta temperatura c'è?

Arduino può leggere queste quantità con i pin analogici.

8.2 Dove sono i pin analogici (A0, A1, …)
Sulla scheda Arduino Nano R4 trovi dei pin con scritto:
A0….A1….A2…
Questi pin sono speciali perché Arduino può usarli come "sensori": leggono un numero che cambia.

*Oggi useremo **"A0"** perché è il primo e più comodo.*

ESERCIZIO 8.2 – Trova A0 e segnalo

Prendi la scheda e cerca il pin A0.

8.3 Il potenziometro: la manopola che cambia valore (lo abbiamo già usato nel volume 1 di ELAB**)**

Per far capire l'analogico serve qualcosa che cambi "piano piano".
Il componente perfetto è il potenziometro (una manopola).

Ha 3 piedini:

un estremo va a 5V
l'altro estremo va a GND

quello in mezzo (centrale) manda un valore "variabile" ad Arduino

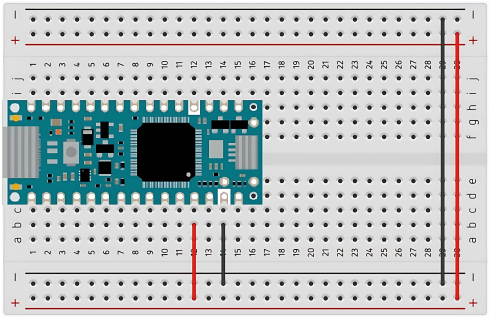
Il piedino centrale è il più importante: è quello che "parla" con A0.

ESERCIZIO 8.3 – Collegare il potenziometro (passo-passo)

Materiale:
Arduino Nano R4
breadboard
1 potenziometro
3 cavetti jumper

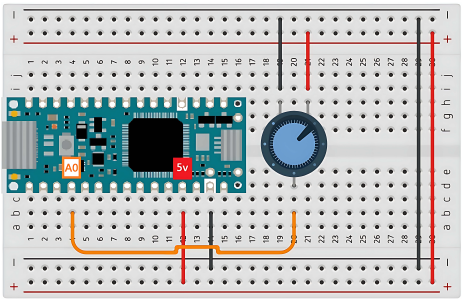
Prepariamo la nostra breadbord

- Inserisci la nostra **Arduino Nano R4** sulla breadboard, come mostrato nell'immagine.
- Alimenta la breadboard collegando il **pin 5V e il pin GND** della scheda alle rispettive linee di alimentazione (positivo e negativo) della breadboard.
- Per facilitare i collegamenti, usa due cavetti (jumper) per collegare tra loro le due linee di alimentazione opposte della breadboard:
- 5V (linea +) da un lato → 5V (linea +) dall'altro lato
- GND (linea −) da un lato → GND (linea −) dall'altro lato
  In questo modo avrai 5V e GND disponibili su entrambi i lati della breadboard.
- Da questo momento in poi, mantieni sempre questi collegamenti anche negli esercizi successivi: è un metodo pratico per avere la breadboard sempre alimentata e collegata correttamente. correttamente



Passaggi

- Inserisci il potenziometro nella breadboard (i 3 piedini in 3 righe diverse). mettilo a cavallo della linea centrale
- Collega un piedino esterno a 5V di Arduino.
- Collega l'altro piedino esterno a GND.
- Collega il piedino centrale a A0.



8.4 La funzione nuova: analogRead()

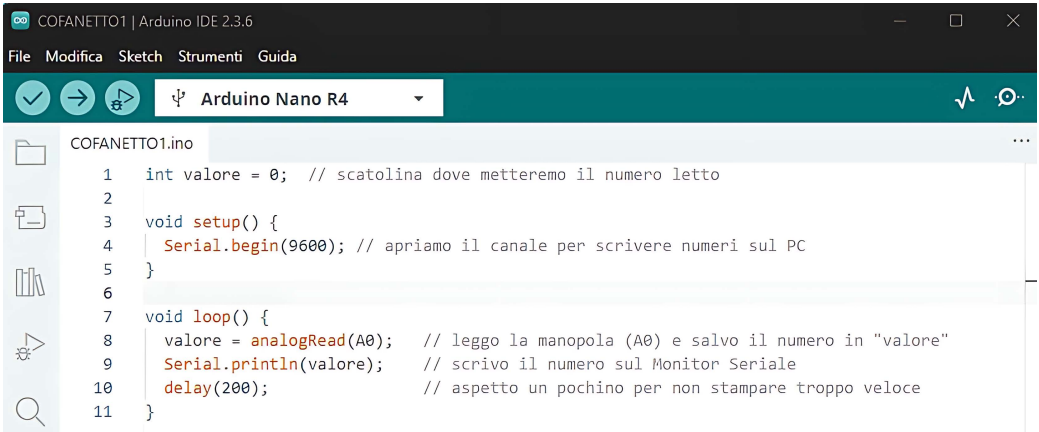
Ora Arduino deve “sentire” la manopola.
La funzione è: **analogRead(A0)**

Questa funzione restituisce un numero:

- **"0 "**quando il segnale è bassissimo (verso GND)
- **"1023 "**quando il segnale è altissimo (verso 5V)

Quindi Arduino non riceve “sì/no”, riceve un numero.

ESERCIZIO 8.4 – Primo programma: leggo un numero e lo salvo in una scatolina (int)




```
1   int valore = 0;  // scatolina dove metteremo il numero letto
2
3   void setup() {
4     Serial.begin(9600); // apriamo il canale per scrivere numeri sul PC
5   }
6
7   void loop() {
8     valore = analogRead(A0);   // leggo la manopola (A0) e salvo il numero in "valore"
9     Serial.println(valore);    // scrivo il numero sul Monitor Seriale
10    delay(200);                // aspetto un pochino per non stampare troppo veloce
11  }
```

RIGA 1: int valore = 0;

"int"

Come abbiamo gia visto significa: numero intero (senza virgola). Serve per creare una "scatolina" capace di contenere numeri come 0, 1, 25, 500, 1023… In pratica: sto dicendo ad Arduino: "mi serve una scatola per numeri".

"valore" : È il nome della scatolina. L'abbiamo chiamata "valore" perché dentro metteremo il valore letto dal pin analogico. Potevi anche chiamarla numero, lettura, pot, ma "valore" è chiaro.

"= " (un uguale) Significa assegna. Vuol dire: "metti dentro la scatolina quello che scrivo dopo".

"0" È il numero che mettiamo dentro all'inizio.Non è obbligatorio per forza, ma è utile perché: così la scatolina parte "pulita"e non contiene numeri casuali

Spiegazione logica della riga 1:

"Creo una scatolina per numeri chiamata valore e ci metto dentro 0."

BLOCCO: void setup() { ... }

"void" significa "vuoto"la funzione non restituisce un risultato: contiene solo comandi.

"setup"è la funzione di preparazione, Arduino la esegue una sola volta quando si accende o si resetta.

Spiegazione logica di setup:

"Qui preparo le cose prima che Arduino inizi a ripetere il programma."

RIGA dentro setup: Serial.begin(9600);

"Serial"

È il nome di un "canale" che Arduino usa per parlare con il computer.

Passa attraverso il cavo USB. È come un walkie-talkie:

Arduino parla il PC ascolta nel Monitor Seriale

".begin" : begin significa "inizia".Quindi: "inizia la comunicazione seriale".

"(9600)" È la velocità della comunicazione.

9600 è un valore standard molto usato.

Importantissimo:

se nel codice hai **Serial.begin(9600)** nel Monitor Seriale devi selezionare *9600 baud* altrimenti vedrai caratteri strani o niente.

Spiegazione logica di setup:

"Arduino, apri il canale per parlare con il PC a velocità 9600."

BLOCCO: void loop() { ... };

"loop" è la funzione che Arduino ripete per sempre finisce e ricomincia.

Spiegazione logica di setup:
"Questi comandi li ripeto all'infinito."

RIGA: valore = analogRead(A0);

Qui Arduino "sente" l'analogico.

"valore" la scatolina che abbiamè il pin analogico dove hai collegato il piedino centrale del potenziometro.o creato prima

"= " assegna: "metti dentro valore il risultato di…"

"analogRead(A0)" è una funzione che legge un pin analogico (A0).

"A0" è il pin analogico dove hai collegato il piedino centraè il pin analogico dove hai collegato il piedino centrale del potenziometro.le del potenziometro.

Cosa restituisce analogRead?

Uun numero tra:

" **0 "** (manopola verso GND)

" 1023 " (manopola verso 5V)

Spiegazione logica della riga:

"Leggo la manopola su A0 e salvo il numero dentro valore."

RIGA: Serial.println(valore);

Ora Arduino "dice" al PC quel numero.

" Serial " stesso canale di prima (Arduino ↔ PC)

" println "

- *"print"* = scrivi
- *"ln"* = "line" → vai a capo

Quindi:
"**valore**" significa: "scrivi quello che c'è dentro la scatolina valore".

Spiegazione logica della riga:
"Scrivo sul Monitor Seriale il numero che ho letto."

RIGA: delay(200);

" **delay "** fai una pausa
" (200) " 200 millisecondi = 0,2 secondi

perché serve?

Perché senza pausa Arduino stamperebbe migliaia di numeri al secondo e il monitor scorrerebbe troppo veloce.

Spiegazione logica della riga:
"Aspetto un pochino prima di leggere e stampare di nuovo."

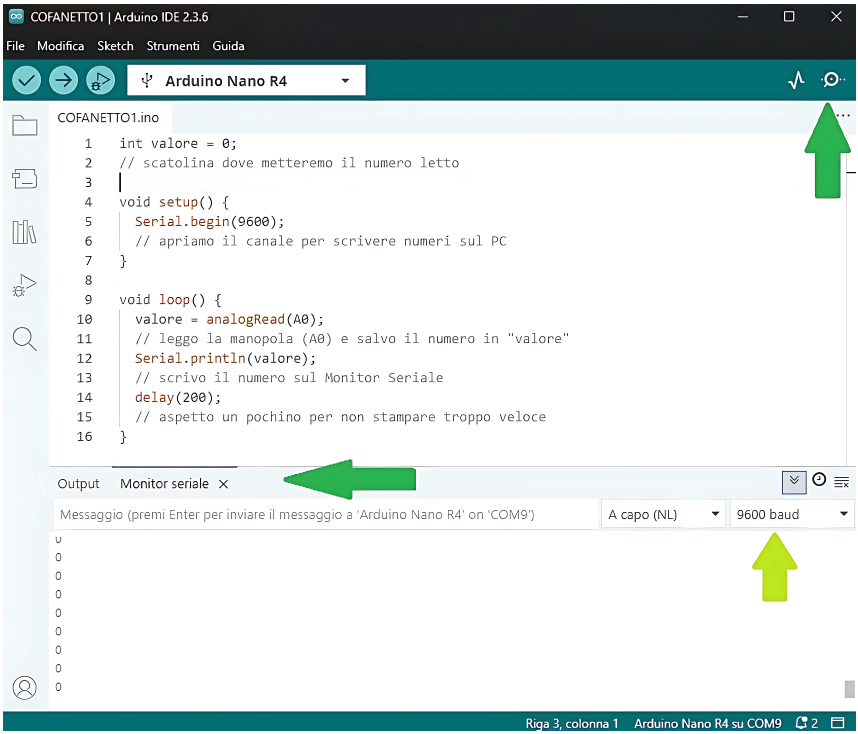
RIGA: delay(200);

" **delay** " fai una pausa
" (200) " 200 millisecondi = 0,2 secondi

perché serve?

Perché senza pausa Arduino stamperebbe migliaia di numeri al secondo e il monitor scorrerebbe troppo veloce.

Spiegazione logica della riga:
"Aspetto un pochino prima di leggere e stampare di nuovo."



Cosa fare sul PC

Carica lo sketch.
Apri Strumenti → Monitor Seriale.
Imposta in basso a destra 9600 baud.
Cosa vedi? Numeri che cambiano mentre giri la manopola.

NOTE

COSA SONO I FOTOTRANSISTOR?

CAPITOLO 10

NOTE

CAPITOLO 11

NOTE

CAPITOLO 12

COSTRUIAMO IL ROBOT SEGUI LUCE

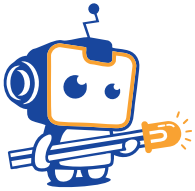
NOTE

NON È FINITA QUI!

Speriamo davvero che ti sia divertito con questi esperimenti per avvicinarti al mondo dell'elettronica!

Nei prossimi volumi espanderemo i concetti visti nei primi due volumi, maggiormente nel dettaglio del funzionamento, utilizzeremo anche altri componenti e soprattutto inizieremo ad addentrarci nel magico mondo della programmazione utilizzando il linguaggio Arduino e le sue schede elettroniche!

Grazie per averci seguito in questa prima avventura e speriamo che gradirete anche le successive!



ELAB